

Searching Algorithms

For additional reading and resources, you can refer to

<https://jeffe.cs.illinois.edu/teaching/algorithms/book/00-intro.pdf> and
<https://jeffe.cs.illinois.edu/teaching/algorithms/book/01-recursion.pdf>

In the morning, we discussed what is an algorithm and the worst-case time complexity.

Linear Search

Let's consider another example where we could use an algorithm. A common problem in programming is that we have to search or find a particular value. We can do this using a searching algorithm.

Let's say we have this array of TAs:

```
tas = ["sosna",
       "rohit",
       "mihret",
       "beniyam",
       "mokhalad",
       "kidist",
       "bontu",
       "kaleb",
       "minase",
       "leena",
       "tadael",
       "admas",
       "simon",
       "nils",
       "betelhem",
       "biniyam",
       "meghan",
       "samuel",
       "amir",
       "matthew",
       "mika",
       "amanuel",
       "hanna",
       "girum",
       "sonika",
       "abdishakur"]

# Let's say we are looking for Biniyam in this list.
# To find him, we can iterate through the list until we have found him

def linear_search(arr, target):
```

```
for i in range(len(arr)):
    if arr[i] == target:
        return i
return -1

# Since Biniyam is in the list, we get the INDEX of where in the list
# he is
print(linear_search(tas, "biniyam"))
15

# Since I am not in the list, we get the INDEX -1
print(linear_search(tas, "huacheng"))
-1
```

Time Complexity of Linear Search

Looking at the algorithm above, we can try to find the time complexity of linear search.

Let's first figure out what is the WORST case for linear search. This means, when will this algorithm take the longest.

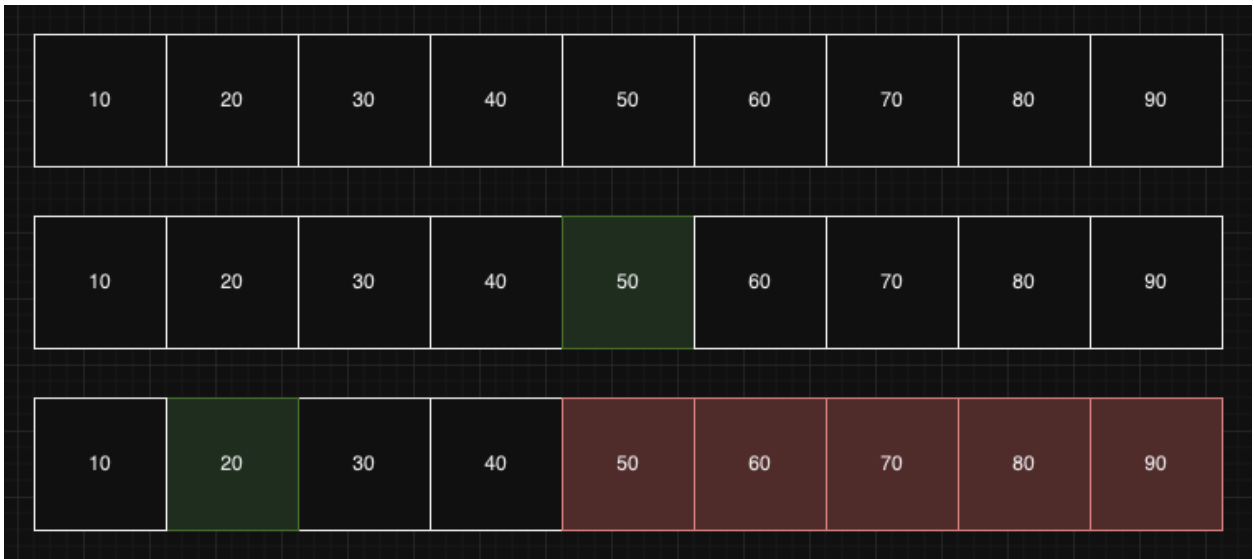
worst-case time complexity of linear search is $O(n)$

Binary Search

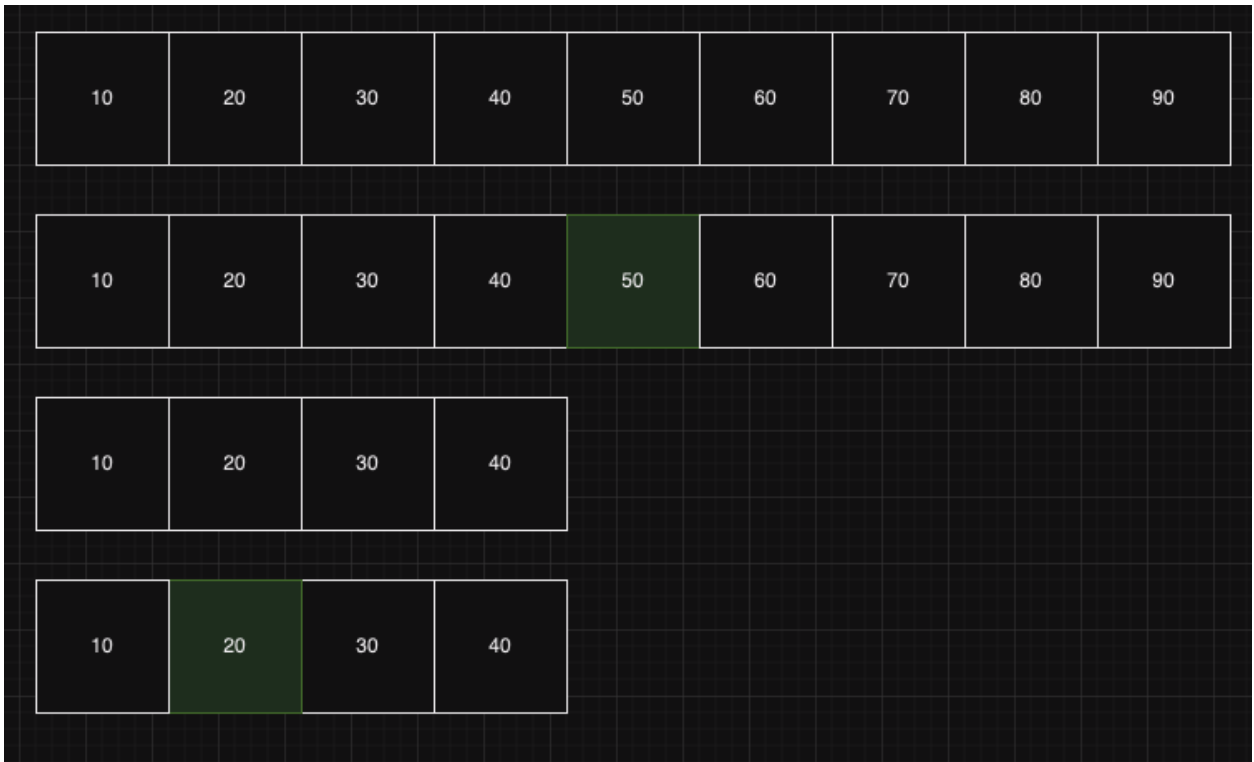
Binary search works by dividing our search space in half. Consider the sorted array below:

```
grades = [10, 20, 30, 40, 50, 60, 70, 80, 90]
```

If we want to find the value, 20, we can do this by taking the element in the middle of the array and comparing it to the value we want to find.



This is conceptually more like:



where we reduce the elements we need to look at.

```
def binary_search(arr, target):  
    left, right = 0, len(arr) - 1 # [left, right] is the range we are  
    focusing on  
    while (left <= right):  
        mid = (left + right) // 2 # compute the index of the middle  
        element  
        if (target == arr[mid]):
```

```
        return mid
    if (target > arr[mid]):
        left = mid + 1
    else:
        right = mid - 1
    return -1

print(binary_search(grades, 80))
7
print(binary_search(grades, 25))
-1
```

Recursive Binary Search

We wrote the above binary search algorithm in an iterative way. However, we've also talked about recursion. The cool thing is that we can also write this binary search function recursively.

How do we do this? Let's think about how we will set up our recursion.

1. What inputs would we need for our recursive function? Keep in mind that for the iterative solution, we have the array and the element we are looking for.
2. A recursive function is made up of 2 parts: the base case and the recursive step. First, let's think about what the recursive step would be, i.e. what generally do we want to do each time?

Recursive step pseudocode

1. Now, what would be the base case?

Base case pseudocode

Now that we have these 3 parts, we can put it together into a recursive function:

```
def binary_search(arr, target, left, right):
    if (left > right):
        return -1
    mid = (left + right) // 2
    if (target == arr[mid]):
        return mid
    if (target > arr[mid]):
        return binary_search(arr, target, mid + 1, right)
    if (target < arr[mid]):
        return binary_search(arr, target, left, mid - 1)

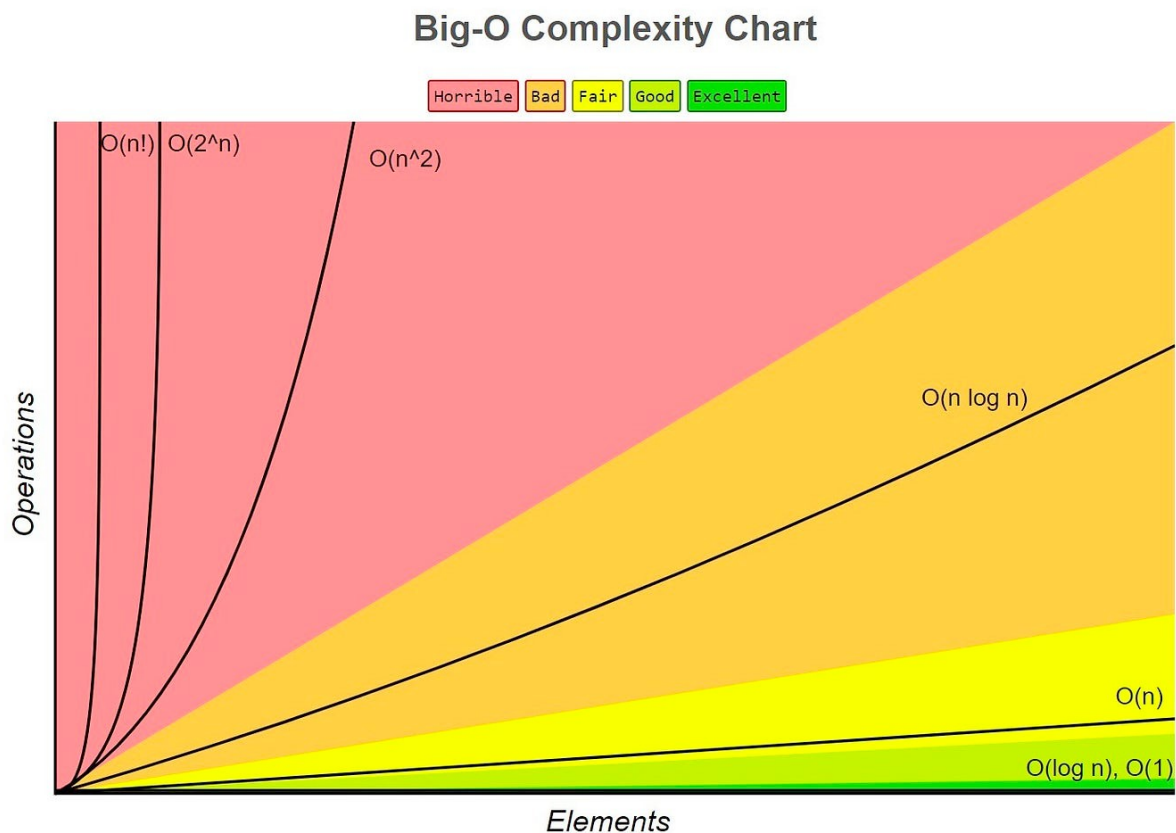
print(binary_search(grades, 80, 0, len(grades) - 1))
print(binary_search(grades, 25, 0, len(grades) - 1))
```

Time Complexity of Binary Search

Binary search reduces the amount of space we have to look through each time. With linear search, in the worst case, we have to look at every element in the array ($O(N)$). However, with binary search, in the worst case, we reduce the range of the array by half in $O(1)$ time.

If the number of elements in the array is N , then with binary search we are looking at $N/2$ elements after first round. In general, every time we are dividing our search space, this ends up being a logarithmic time complexity $O(\log(N))$. This is because by dividing our space, we are reducing the time complexity to be less than $O(N)$.

Recall the graph shown from this morning:



In this graph, we can see $O(\log(N))$ grows slower than $O(N)$.

We will see more algorithms where we divide our search space in the coming days.

Binary Search Constraints

Why does binary search work? It only works when we are given a SORTED array or list. When the array or list is sorted, we are guaranteed that all elements to the right of X are greater than X and all elements to the left of X are less than X.

What do you think would happen if we tried binary search on an unsorted array?

```
unsorted_arr = [7, 2, 6, 0, 18, 4, 9]
print(binary_search(unsorted_arr, 6, 0, len(unsorted_arr) - 1))
-1
```